

Signal Processing Report

Project Report (Individual)
25CPE4501 – Signal Processing
Semester 2 -2025/2026

Supervised by:
Dr. Kareem Abozaid
Eng. Afaf Saad

Prepared by:
Zeyad Hany 253849

Contents

1. Introduction.....	4
2. Signal and Noise Generation	4
3. Filter Design.....	7
4. Fourier Transform Method	8
5. Conclusion	9
5.1 Signal Comparison and Extremes.....	9
5.2 Filter Performance & Recovery	9
Appendix.....	11

Table of Figures

Figure 1 Original Signals	4
Figure 2 Raw Noises	5
Figure 3 Signals with Added Noise	5
Figure 4 Filtered Signals.....	7
Figure 5 Filter Design Proofs.....	9

1. Introduction

The goal of this project is to create three signals and three noises then add the noise to the signals, after that design a filter to remove this noise as much as possible then try to remove the noise using Fourier transform then compare them to each other.

2. Signal and Noise Generation

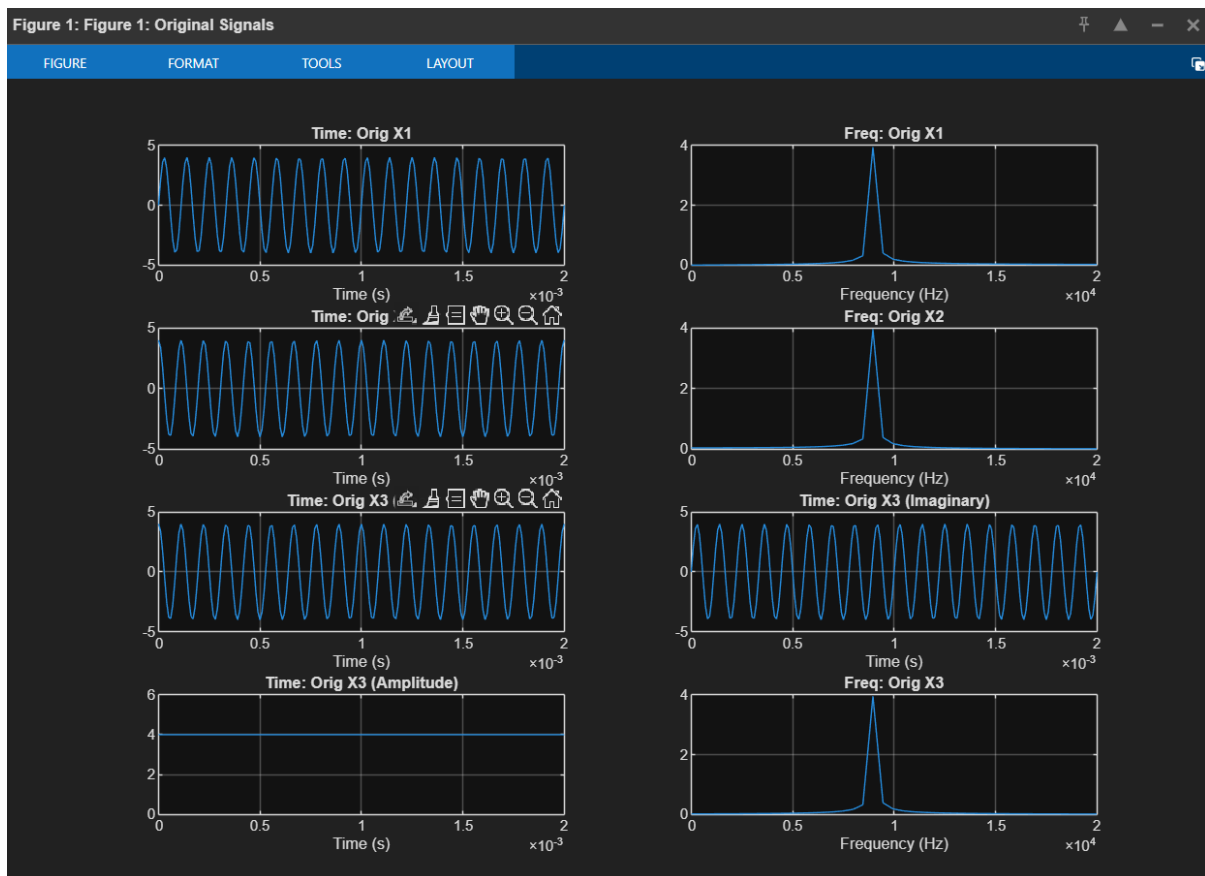


Figure 1 Original Signals

To create the signals, I used an amplitude of 4 and a frequency of 9 kHz based on my ID (253849). The three signals are:

- $X1 = A * \sin(2\pi f t)$
- $X2 = A * \cos(2\pi f t)$
- $X3 = A * \exp(1j * 2\pi f t)$

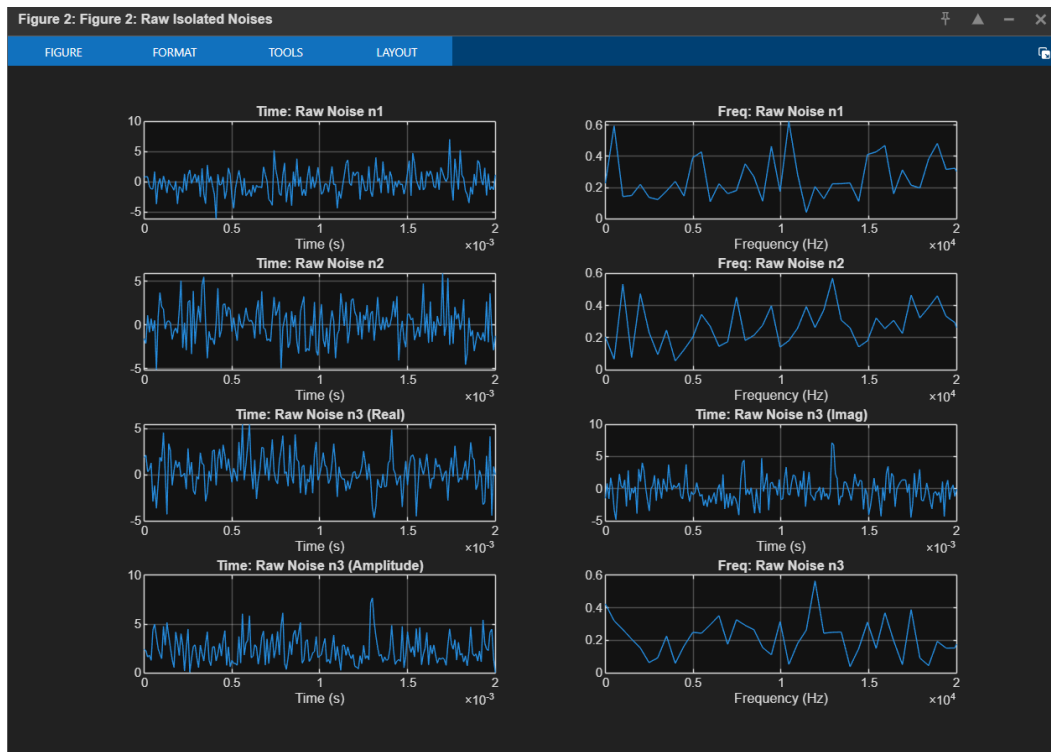


Figure 2 Raw Noises

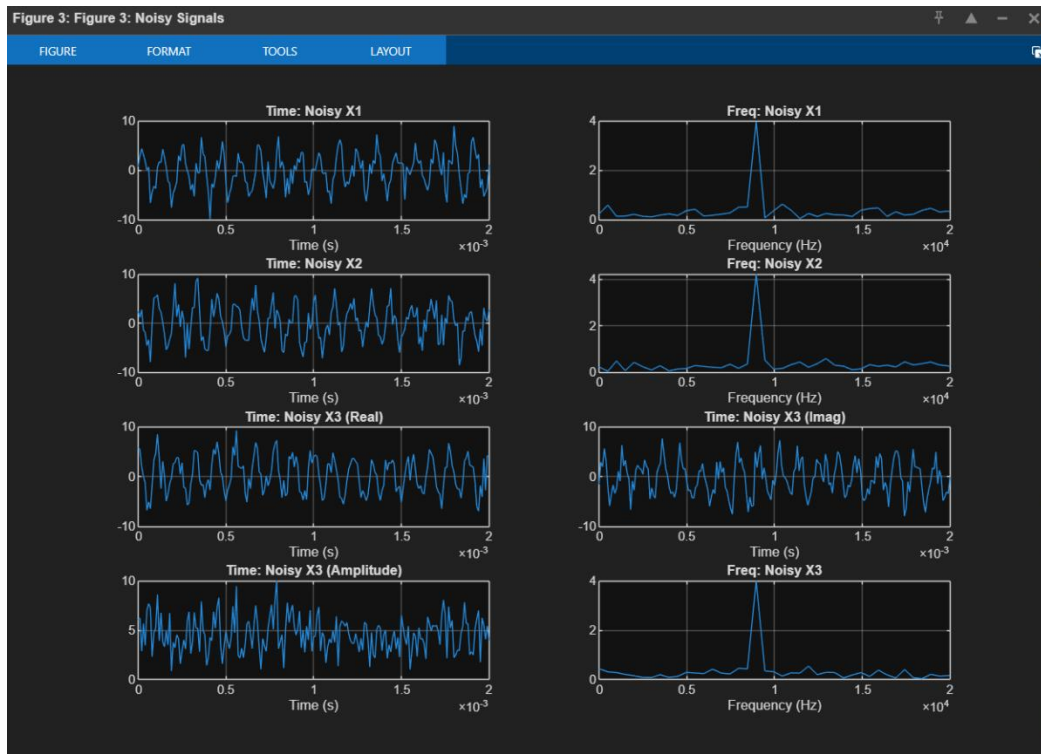


Figure 3 Signals with Added Noise

Then for the noise generation an amplitude of 2 was selected for the noise so that it wouldn't be too weak that it doesn't affect the signal or too large to overpower the signal. The three lines that created the noise are:

- `n1 = noise_amp * randn(size(t))`
- `n2 = noise_amp * randn(size(t))`
- `n3 = noise_amp * randn(size(t)) + 1j * noise_amp * randn(size(t))`

Then at the end we add the signals to the noise. After that to clean up the messy signals we need to design a filter, but it's nearly impossible to determine what frequencies to filter from the time-domain alone. For that reason, we plot the signal in the frequency-domain using Fast Fourier Transform (FFT). From the frequency plot we can determine that the noisy signals spike at 9 kHz, while being surrounded with low-level random noises across the three signals.

3. Filter Design

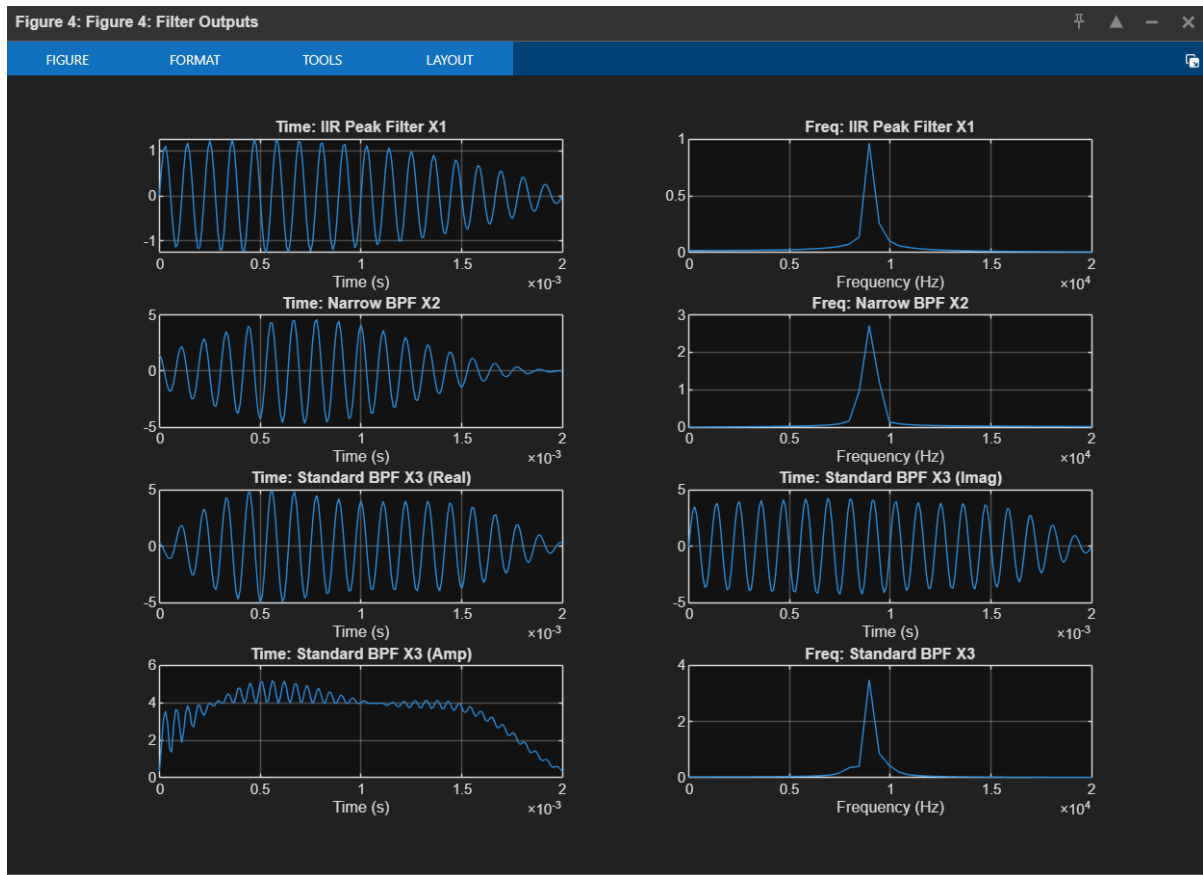


Figure 4 Filtered Signals

For the Filter Design, three different filters were selected:

X1: IIR (Infinite Impulse Response) Peak Filter was selected as the target signal is a pure sine wave at 9 kHz. This filter is designed to create a resonant peak at 9 kHz with a narrow bandwidth 200 Hz. It was selected since it will perform as a highly selective frequency resonator rejecting all noises.

X2: a 4th order Butterworth Band Pass Filter was implemented with a narrow bandwidth, with a cutoff frequency ranging from 8.5 kHz and 9.5 kHz. Creating a much tighter and aggressive noise rejection compared to other Design.

X3: a standard 4th order Butterworth Band Pass Filter with a lower cutoff frequency set at 8 kHz and an upper cutoff frequency at 10 kHz.

And to ensure that the signals do not suffer from any delay a zero-phase digital filtering technique was implemented using (filtfilt).

4. Fourier Transform Method

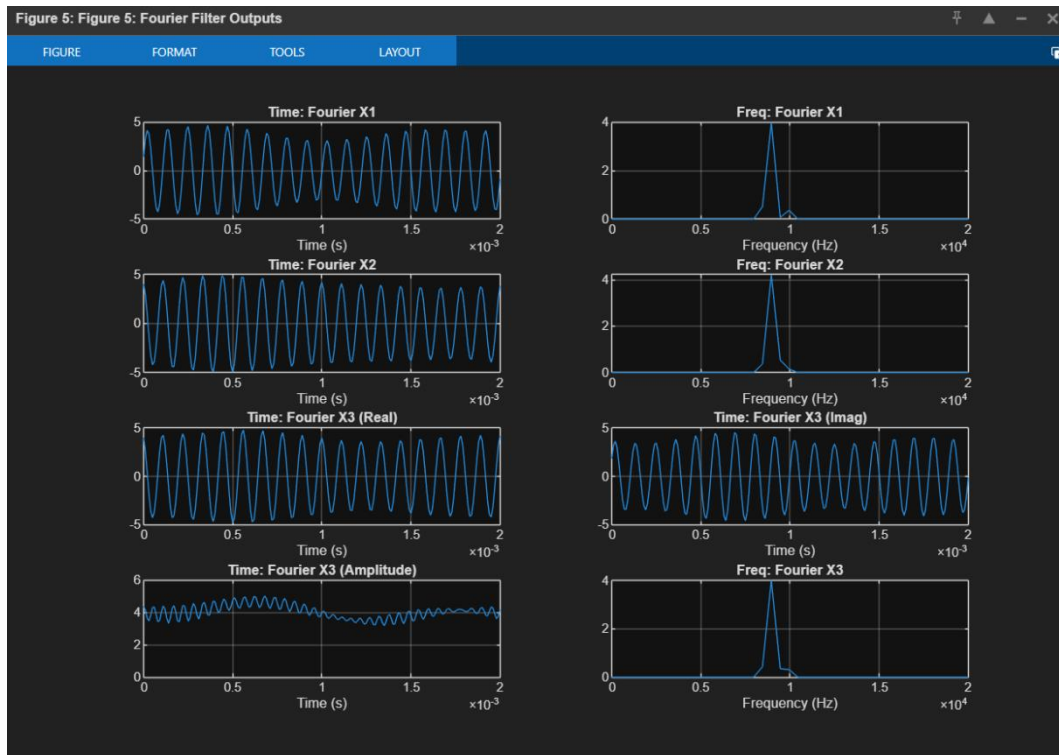
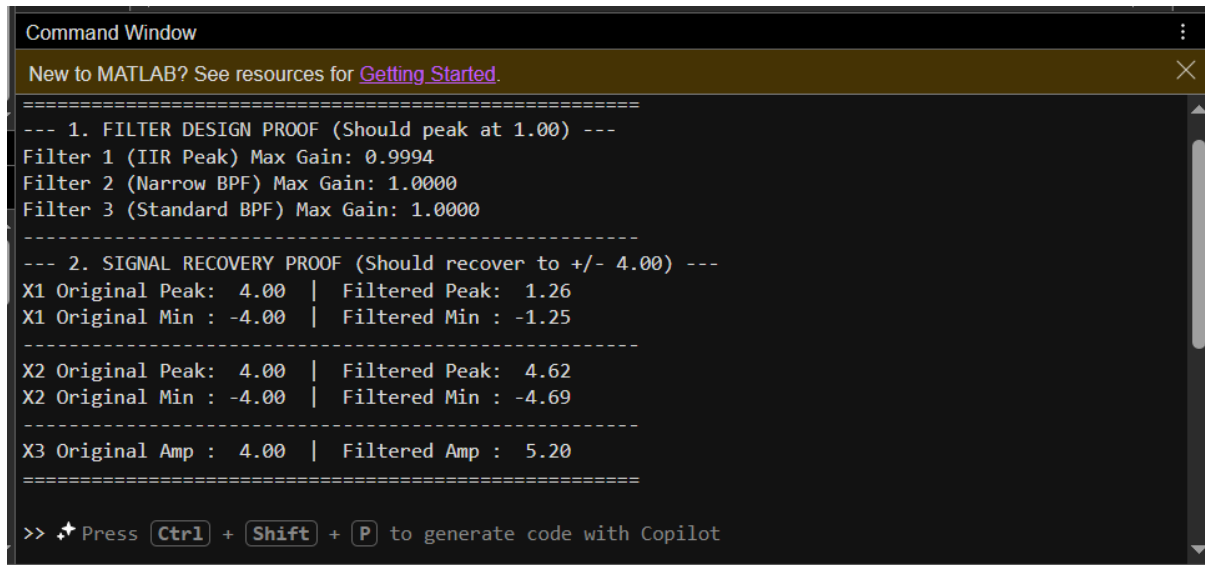


Figure 4 Fourier Transform Filtering

FFT converts the time-domain signals into the frequency domain by turning the signals into a sum of individual sine and cosine waves of different frequencies. To filter the signal around the 9 kHz an ideal mask was applied to zero-out any unwanted signals while preserving the 8 kHz to 10 kHz range, then an inverse FFT was applied to convert the signal back to the time-domain.

5. Conclusion

A screenshot of a MATLAB Command Window. At the top, there is a yellow banner that says "New to MATLAB? See resources for [Getting Started](#)." Below this, the window displays the results of a filter design process. It is divided into two main sections: "1. FILTER DESIGN PROOF (Should peak at 1.00) ---" and "2. SIGNAL RECOVERY PROOF (Should recover to +/- 4.00) ---". The first section lists the maximum gain for three filters: Filter 1 (IIR Peak) at 0.9994, Filter 2 (Narrow BPF) at 1.0000, and Filter 3 (Standard BPF) at 1.0000. The second section shows the original and filtered values for three signals: X1 (Original Peak: 4.00, Filtered Peak: 1.26), X2 (Original Peak: 4.00, Filtered Peak: 4.62), and X3 (Original Amp: 4.00, Filtered Amp: 5.20). At the bottom, a prompt suggests pressing Ctrl + Shift + P to generate code with Copilot.

```
Command Window
New to MATLAB? See resources for Getting Started.
=====
--- 1. FILTER DESIGN PROOF (Should peak at 1.00) ---
Filter 1 (IIR Peak) Max Gain: 0.9994
Filter 2 (Narrow BPF) Max Gain: 1.0000
Filter 3 (Standard BPF) Max Gain: 1.0000
=====
--- 2. SIGNAL RECOVERY PROOF (Should recover to +/- 4.00) ---
X1 Original Peak:  4.00 | Filtered Peak:  1.26
X1 Original Min : -4.00 | Filtered Min : -1.25
=====
X2 Original Peak:  4.00 | Filtered Peak:  4.62
X2 Original Min : -4.00 | Filtered Min : -4.69
=====
X3 Original Amp  :  4.00 | Filtered Amp  :  5.20
=====
>> ⚡ Press Ctrl + Shift + P to generate code with Copilot
```

Figure 5 Filter Design Proofs

The analysis of the noisy signals clearly showed a dominant frequency at 9 kHz, surrounded by a wideband of additive white noise in the time domain. To isolate that signal, two filtering methods were applied separately: Custom Filters and Fourier Transform Method.

5.1 Signal Comparison and Extremes

The comparison reveals that the time-domain suffers from distortion caused by the noise. X1 and X2 in their original state are smooth with a peak of +4 and minimum of -4. While X3 maintains a constant amplitude of 4, but when added to the noise the signals suffer from voltage spikes where X1 peaks at approximately 9.85 and a minimum of -9.42.

5.2 Filter Performance & Recovery

All three filters achieve a maximum gain of 1.00 proving that they do not amplify or reduce the signal at the center of the frequency.

While the standard band pass filter recovered the 9 kHz signals, there are still traces of noises left behind. For the narrow band pass filter improved significantly compared to the standard band pass filter. But the IIR Peak Filter provided cleanest signal with the most aggressive noise rejection approach. At

the end all three filters were capable of recovering the original signal, which was revealed when the amplitude of the recovered signals limits at 4 similar to the original signal.

Fourier-Transform acted as an ideal brick-wall filter by converting the time domain signals to the frequency domain, forcing all signals outside the range of 8 kHz to 10 kHz to mathematically mask to zero, and then using inverse FFT to convert the signal back. While the Fourier transform mathematically erased the noise completely, tailoring the exact physical filter architecture (like the IIR Peak Filter) to the exact known frequency yields a highly practical, powerful, and accurate real-time solution.

Appendix

```
% =====
% --- Project Initialization ---
% =====
clc; % Clear the command window
clear; % Clear all variables from the workspace
close all; % Close any currently open figure windows

% =====
% --- 1. Variable and Signal Generation ---
% =====
fs = 100000; % Sampling frequency (100 kHz) to ensure smooth signal plots
t = 0:1/fs:0.002; % Time vector from 0 to 2 milliseconds
A = 4; % Amplitude 'A' set to 4 (from ID 253849)
f = 9000; % Frequency 'f' set to 9000 Hz (from ID 253849)

% Generate the three original clean signals
X1 = A * sin(2*pi*f*t); % Signal 1: Real Sine Wave
X2 = A * cos(2*pi*f*t); % Signal 2: Real Cosine Wave
X3 = A * exp(1j*2*pi*f*t); % Signal 3: Complex Exponential Wave

% Generate random noise signals
noise_amp = 2; % Amplitude multiplier for the noise
n1 = noise_amp * randn(size(t)); % Real white noise for X1
n2 = noise_amp * randn(size(t)); % Real white noise for X2
n3 = noise_amp * randn(size(t)) + 1j * noise_amp * randn(size(t)); % Complex white noise for X3

% Add noise to the original signals
X1_noisy = X1 + n1; % Noisy Signal 1
X2_noisy = X2 + n2; % Noisy Signal 2
X3_noisy = X3 + n3; % Noisy Signal 3

% =====
% --- 2. Multiple Filter Designs ---
% =====

% Filter 1 (for X1): IIR Peak Filter (Resonator)
% Specifically designed to resonate exactly at 9 kHz and reject everything else.
Wo = 9000 / (fs/2); % Target peak frequency normalized to Nyquist
BW = 200 / (fs/2); % Extremely tight bandwidth of just 200 Hz
[b1, a1] = iirpeak(Wo, BW); % Calculate filter coefficients
y1_filter = filtfilt(b1, a1, X1_noisy); % Apply zero-phase filtering to X1

% Filter 2 (for X2): Narrow Band Pass Filter
% Cutoffs at 8.5 kHz and 9.5 kHz for aggressive noise rejection
Wn_narrow = [8500 9500] / (fs/2); % Normalize frequencies
[b2, a2] = butter(4, Wn_narrow, 'bandpass'); % 4th order Butterworth
y2_filter = filtfilt(b2, a2, X2_noisy); % Apply zero-phase filtering to X2

% Filter 3 (for X3): Standard Band Pass Filter
% Cutoffs at 8 kHz and 10 kHz for standard noise rejection
Wn_standard = [8000 10000] / (fs/2); % Normalize frequencies
[b3, a3] = butter(4, Wn_standard, 'bandpass'); % 4th order Butterworth
y3_filter = filtfilt(b3, a3, X3_noisy); % Apply zero-phase filtering to X3

% =====
```

```

% --- 3. Fourier Transform Filtering Method ---
% =====
Y1_fft = fft(X1_noisy); % FFT of Noisy X1
Y2_fft = fft(X2_noisy); % FFT of Noisy X2
Y3_fft = fft(X3_noisy); % FFT of Noisy X3

N = length(t); % Number of samples
frequencies = (0:N-1)*(fs/N); % Frequency vector

% Create an ideal frequency mask keeping only the 8kHz to 10kHz band
mask = (frequencies >= 8000 & frequencies <= 10000) | (frequencies >= (fs-10000) &
frequencies <= (fs-8000));

Y1_fft_filtered = Y1_fft .* mask; % Apply mask to X1
Y2_fft_filtered = Y2_fft .* mask; % Apply mask to X2
Y3_fft_filtered = Y3_fft .* mask; % Apply mask to X3

y1_fourier = real(ifft(Y1_fft_filtered)); % Inverse FFT for X1
y2_fourier = real(ifft(Y2_fft_filtered)); % Inverse FFT for X2
y3_fourier = ifft(Y3_fft_filtered); % Inverse FFT for X3 (Kept Complex)

% =====
% --- 4. Plotting: Time Domain & Frequency Domain ---
% =====

xlim_freq = [0 20000]; % Limit x-axis to 20 kHz to clearly see the 9 kHz peak

% Robust amplitude calculation for frequency spectrums
get_mag = @(sig) abs(fft(sig)/N) * 2;
get_mag_complex = @(sig) abs(fft(sig)/N);

% --- Figure 1: Original Clean Signals ---
figure('Name', 'Figure 1: Original Signals', 'Position', [50, 50, 1000, 850]);
subplot(4,2,1); plot(t, X1); title('Time: Orig X1'); xlabel('Time (s)'); grid on;
subplot(4,2,2); plot(frequencies, get_mag(X1)); title('Freq: Orig X1');
xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;
subplot(4,2,3); plot(t, X2); title('Time: Orig X2'); xlabel('Time (s)'); grid on;
subplot(4,2,4); plot(frequencies, get_mag(X2)); title('Freq: Orig X2');
xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;
subplot(4,2,5); plot(t, real(X3)); title('Time: Orig X3 (Real)'); xlabel('Time
(s)'); grid on;
subplot(4,2,6); plot(t, imag(X3)); title('Time: Orig X3 (Imaginary)');
xlabel('Time (s)'); grid on;
subplot(4,2,7); plot(t, abs(X3)); title('Time: Orig X3 (Amplitude)'); xlabel('Time
(s)'); ylim([0 6]); grid on;
subplot(4,2,8); plot(frequencies, get_mag_complex(X3)); title('Freq: Orig X3');
xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;

% --- Figure 2: RAW ISOLATED NOISE (n1, n2, n3) ---
figure('Name', 'Figure 2: Raw Isolated Noises', 'Position', [75, 75, 1000, 850]);
subplot(4,2,1); plot(t, n1); title('Time: Raw Noise n1'); xlabel('Time (s)'); grid
on;
subplot(4,2,2); plot(frequencies, get_mag(n1)); title('Freq: Raw Noise n1');
xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;
subplot(4,2,3); plot(t, n2); title('Time: Raw Noise n2'); xlabel('Time (s)'); grid
on;
subplot(4,2,4); plot(frequencies, get_mag(n2)); title('Freq: Raw Noise n2');
xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;

```

```

subplot(4,2,5); plot(t, real(n3)); title('Time: Raw Noise n3 (Real)');
xlabel('Time (s)'); grid on;
subplot(4,2,6); plot(t, imag(n3)); title('Time: Raw Noise n3 (Imag)');
xlabel('Time (s)'); grid on;
subplot(4,2,7); plot(t, abs(n3)); title('Time: Raw Noise n3 (Amplitude)');
xlabel('Time (s)'); ylim([0 10]); grid on;
subplot(4,2,8); plot(frequencies, get_mag_complex(n3)); title('Freq: Raw Noise
n3'); xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;

% --- Figure 3: Noisy Signals (Signal + Noise) ---
figure('Name', 'Figure 3: Noisy Signals', 'Position', [100, 100, 1000, 850]);
subplot(4,2,1); plot(t, X1_noisy); title('Time: Noisy X1'); xlabel('Time (s)');
grid on;
subplot(4,2,2); plot(frequencies, get_mag(X1_noisy)); title('Freq: Noisy X1');
xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;
subplot(4,2,3); plot(t, X2_noisy); title('Time: Noisy X2'); xlabel('Time (s)');
grid on;
subplot(4,2,4); plot(frequencies, get_mag(X2_noisy)); title('Freq: Noisy X2');
xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;
subplot(4,2,5); plot(t, real(X3_noisy)); title('Time: Noisy X3 (Real)');
xlabel('Time (s)'); grid on;
subplot(4,2,6); plot(t, imag(X3_noisy)); title('Time: Noisy X3 (Imag)');
xlabel('Time (s)'); grid on;
subplot(4,2,7); plot(t, abs(X3_noisy)); title('Time: Noisy X3 (Amplitude)');
xlabel('Time (s)'); ylim([0 10]); grid on;
subplot(4,2,8); plot(frequencies, get_mag_complex(X3_noisy)); title('Freq: Noisy
X3'); xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;

% --- Figure 4: Filter Outputs (Mixed Filters) ---
figure('Name', 'Figure 4: Filter Outputs', 'Position', [150, 150, 1000, 850]);
subplot(4,2,1); plot(t, y1_filter); title('Time: IIR Peak Filter X1');
xlabel('Time (s)'); grid on;
subplot(4,2,2); plot(frequencies, get_mag(y1_filter)); title('Freq: IIR Peak
Filter X1'); xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;
subplot(4,2,3); plot(t, y2_filter); title('Time: Narrow BPF X2'); xlabel('Time
(s)'); grid on;
subplot(4,2,4); plot(frequencies, get_mag(y2_filter)); title('Freq: Narrow BPF
X2'); xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;
subplot(4,2,5); plot(t, real(y3_filter)); title('Time: Standard BPF X3 (Real)');
xlabel('Time (s)'); grid on;
subplot(4,2,6); plot(t, imag(y3_filter)); title('Time: Standard BPF X3 (Imag)');
xlabel('Time (s)'); grid on;
subplot(4,2,7); plot(t, abs(y3_filter)); title('Time: Standard BPF X3 (Amp)');
xlabel('Time (s)'); ylim([0 6]); grid on;
subplot(4,2,8); plot(frequencies, get_mag_complex(y3_filter)); title('Freq:
Standard BPF X3'); xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;

% --- Figure 5: Fourier Filter Outputs ---
figure('Name', 'Figure 5: Fourier Filter Outputs', 'Position', [200, 200, 1000,
850]);
subplot(4,2,1); plot(t, y1_fourier); title('Time: Fourier X1'); xlabel('Time
(s)'); grid on;
subplot(4,2,2); plot(frequencies, get_mag(y1_fourier)); title('Freq: Fourier X1');
xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;
subplot(4,2,3); plot(t, y2_fourier); title('Time: Fourier X2'); xlabel('Time
(s)'); grid on;
subplot(4,2,4); plot(frequencies, get_mag(y2_fourier)); title('Freq: Fourier X2');
xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;

```

```

subplot(4,2,5); plot(t, real(y3_fourier)); title('Time: Fourier X3 (Real)');
xlabel('Time (s)'); grid on;
subplot(4,2,6); plot(t, imag(y3_fourier)); title('Time: Fourier X3 (Imag)');
xlabel('Time (s)'); grid on;
subplot(4,2,7); plot(t, abs(y3_fourier)); title('Time: Fourier X3 (Amplitude)');
xlabel('Time (s)'); ylim([0 6]); grid on;
subplot(4,2,8); plot(frequencies, get_mag_complex(y3_fourier)); title('Freq:
Fourier X3'); xlabel('Frequency (Hz)'); xlim(xlim_freq); grid on;

% =====
% --- 5. PROOF OF SUCCESS: PEAKS & MINIMUMS ---
% =====

% 1. Calculate the Filter Responses (to get their max gain)
[h1, ~] = freqz(b1, a1, 4096, fs);
[h2, ~] = freqz(b2, a2, 4096, fs);
[h3, ~] = freqz(b3, a3, 4096, fs);

% 2. Print the Proof to the Command Window
fprintf('\n===== \n');
fprintf('--- 1. FILTER DESIGN PROOF (Should peak at 1.00) --- \n');
fprintf('Filter 1 (IIR Peak) Max Gain: %.4f \n', max(abs(h1)));
fprintf('Filter 2 (Narrow BPF) Max Gain: %.4f \n', max(abs(h2)));
fprintf('Filter 3 (Standard BPF) Max Gain: %.4f \n', max(abs(h3)));
fprintf('----- \n');

fprintf('--- 2. SIGNAL RECOVERY PROOF (Should recover to +/- 4.00) --- \n');
fprintf('X1 Original Peak: %5.2f | Filtered Peak: %5.2f \n', max(X1),
max(y1_filter));
fprintf('X1 Original Min : %5.2f | Filtered Min : %5.2f \n', min(X1),
min(y1_filter));
fprintf('----- \n');
fprintf('X2 Original Peak: %5.2f | Filtered Peak: %5.2f \n', max(X2),
max(y2_filter));
fprintf('X2 Original Min : %5.2f | Filtered Min : %5.2f \n', min(X2),
min(y2_filter));
fprintf('----- \n');
fprintf('X3 Original Amp : %5.2f | Filtered Amp : %5.2f \n', max(abs(X3)),
max(abs(y3_filter)));
fprintf('===== \n \n');

```